OPERATING SYSTEMS PROJECT
REVIEW 2 — METHODOLOGY AND PARTIAL IMPLEMENTATION (LAB ASSESSMENT 9)

# Event-Driven Runtime-Adaptive CPU Scheduling

## with Sequential Tabular Q-Learning

Single-CPU discrete-event simulator • causal arrived-work state abstraction • sequential tabular Q-learning • seven seeded workload families • held-out crossed-bootstrap evaluation

| Field | Value |
|---|---|
| Project title | Event-Driven Runtime-Adaptive CPU Scheduling with Sequential Q-Learning |
| Review stage | Review 2 — Methodology and Partial Implementation (Lab Assessment 9) |
| Repository | github.com/lostpilot404/OS-Project |
| Language / runtime | Python 3.11.2 (NumPy 2.4.6, pandas 3.0.6, Matplotlib 3.11.2) |
| Automated verification | 273 checks, all passing (re-run during this build) |
| Dataset | 7 synthetic families; 6,000 train / 140 validation / 210 held-out workloads |
| Report generated | 11 October 2026 |

| 6 / 6 | 3,150 | 6,000 | 210 | 162 × 4 | 273 |
|---|---|---|---|---|---|
| major modules implemented | processes in the held-out dataset | training workloads | held-out test workloads | causal states × actions | automated tests passing |

**Summary of the work presented.** The proposed system replaces a fixed CPU scheduling policy with a causal, event-driven runtime controller. A single-CPU discrete-event simulator keeps one persistent FIFO ready queue and, at every dispatch or Round-Robin quantum boundary, asks a controller to choose FCFS, SJF, Round Robin or Priority for the next execution segment of the same evolving trace. The controller may observe only work that has already arrived; five causal features are discretised into 162 states and a tabular Q-learning agent with an undiscounted interval reward ($\Sigma r = -$total waiting time / q) learns when to switch. This report documents the methodology (§1), the architecture (§2), the six modules and their workflows (§3–4), the generated dataset (§5), the tools and algorithms (§6), the experimental plan and metrics (§7), the implementation completed so far with its outputs (§8), and the challenges and remaining plan (§9). Current measured status on the 210 held-out workloads: runtime Q-learning 104.789 mean waiting time versus 215.729 for Round Robin and 93.345 for SJF, with 273/273 automated checks passing.

# Contents

The report follows the Review 2 rubric item by item; the nine headings below are the nine required items.

**Contents** 1

**1. Detailed methodology of the proposed system** 3

1.1 Research questions . . . 3

1.2 Methodology pipeline . . . 3

1.3 Causal observation contract . . . 3

1.4 State abstraction and learning objective . . . 4

1.5 Methodological controls . . . 4

**2. Overall conceptual, architectural and pipeline diagram** 5

2.1 Runtime control flow . . . 5

**3. Description of the major modules** 7

M1 — Workload generator and dataset builder . . . 7

M2 — Discrete-event runtime environment . . . 7

M3 — Causal observation and state encoder . . . 7

M4 — Sequential Q-learning controller . . . 7

M5 — Predeclared heuristic controller . . . 7

M6 — Experiment orchestration, metrics and reporting . . . 8

**4. Workflow diagram for each major module** 9

M1. Workload generator and dataset builder . . . 9

M2. Discrete-event runtime environment . . . 9

M3. Causal observation and state encoder . . . 10

M4. Sequential Q-learning controller . . . 11

M5. Predeclared heuristic controller . . . 12

M6. Experiment orchestration, metrics and reporting . . . 12

**5. Dataset details and data collection procedure** 14

5.1 Data collection procedure . . . 14

5.2 Dataset inventory . . . 14

5.3 Family specification . . . 14

5.4 Measured dataset statistics . . . 14

5.5 Data integrity and assumptions . . . 15

**6. Tools, technologies, algorithms and frameworks used** 16

6.1 Tools and technologies . . . 16

6.2 Algorithms . . . 16

6.3 Controller pseudocode . . . 16

6.4 Frameworks and why none of the heavy ones . . . 16

**7. Experimental plan and evaluation metrics** 18

7.1 Experimental plan . . . 18
7.2 Evaluation metrics . . . 18
7.3 Statistical procedure . . . 18
7.4 Results obtained with this plan . . . 18
7.5 What the plan has established so far . . . 20
**8. Implementation progress** **21**
8.1 Outputs demonstrated by each module . . . 21
8.2 Demonstration 1 — automated verification . . . 21
8.3 Demonstration 2 — end-to-end run output . . . 22
8.4 Demonstration 3 — learned behaviour and artifacts . . . 22
**9. Challenges encountered and plan for completing the remaining work** **24**
9.1 Challenges encountered . . . 24
9.2 Plan for completing the remaining work . . . 24
9.3 Sequence for the remaining work . . . 24

# 1. Detailed methodology of the proposed system

**The proposed system** is a single-CPU, event-driven, runtime-adaptive CPU scheduler. Instead of committing one policy for the lifetime of a workload, it re-decides at every dispatch or Round-Robin quantum boundary which of four classical policies should govern the next execution segment of the same evolving trace, and does so from a strictly causal view of the machine. The rest of this section states the questions the system is built to answer, the pipeline that implements it, the information contract that keeps it honest, the state abstraction and objective it learns from, and the controls applied to every comparison.

## 1.1 Research questions

| ID | Question |
|---|---|
| RQ1 | Can a single-CPU scheduler switch between FCFS, SJF, Round Robin and Priority at run time, using only information available at that instant? |
| RQ2 | Does such switching reduce mean waiting time relative to the fixed policies it switches between, under a declared context-switch cost? |
| RQ3 | Does a learned switching policy add value over a hand-written adaptive rule that sees the same observation? |
| RQ4 | Do the gains, if any, survive on a workload family that was never seen in training, and are they larger than the paired bootstrap noise? |

## 1.2 Methodology pipeline

The method is a closed loop between a discrete-event environment that owns the future and a controller that may see only the past and present. Stages 3–7 run once per decision epoch inside a single trace; stage 8 runs once per experiment.

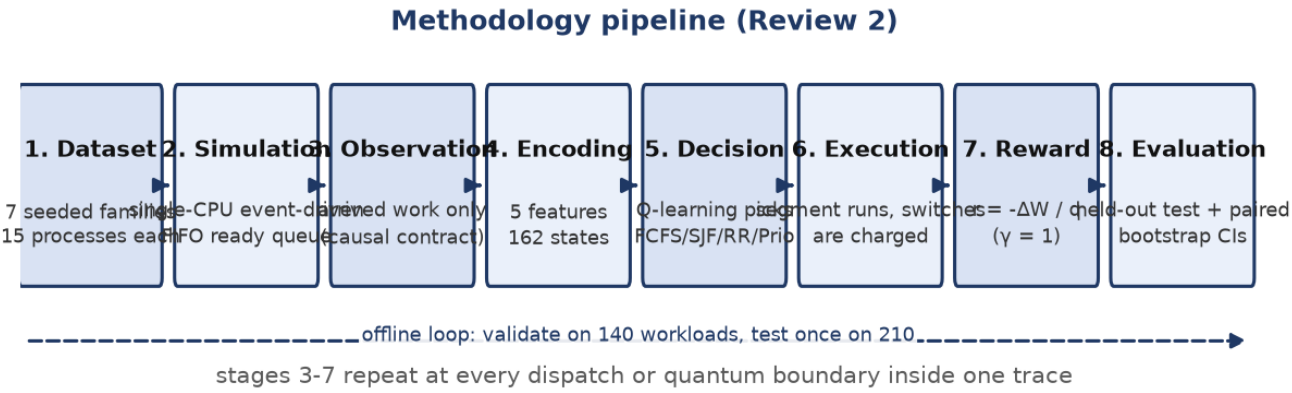



*Figure 1.1: Methodology pipeline. Stages 3–7 form the online decision loop; stage 8 is the offline evaluation that produces every number reported here.*

| Stage | Implemented in | What happens | Output |
|---|---|---|---|
| 1. Dataset | workload/generator.py | Produce seven seeded families of 15-process workloads with SHA-256 derived seeds. | RuntimeWorkload list + manifest |
| 2. Simulation | scheduler/runtime.py | Advance one CPU: arrivals, one FIFO ready queue, remaining bursts, switch cost. | decision epochs + schedule trace |
| 3. Observation | scheduler/runtime.py | Freeze an immutable snapshot of arrived/ready work only (causal contract). | RuntimeObservation |
| 4. Encoding | rl/runtime_state.py | Discretise five causal features into 162 tabular states (3×3×3×3×2). | state index $s \in [0, 162)$ |
| 5. Decision | rl/runtime_controller.py | $\varepsilon$-greedy (training) or greedy (evaluation) choice over four policy actions; explicit fallback in unseen states. | action $a \in$ {FCFS, SJF, RR, Priority} |
| 6. Execution | scheduler/runtime.py | Run the selected segment (RR bounded by the quantum), charge 1 unit when the running PID changes, settle endpoint arrivals. | advanced simulated time |
| 7. Reward | scheduler/runtime.py | Measure the waiting-time area $\Delta W$ accumulated by ready processes; return $r = -\Delta W / q$ ($\gamma = 1$). | interval reward $r_t$ |
| 8. Evaluation | experiments/runtime_experiment.py | Greedy read-only runs on held-out workloads; paired differences with a family-stratified crossed bootstrap. | metric tables + CIs |

*Table 1.1: The eight methodology stages and the module that owns each.*

## 1.3 Causal observation contract

The environment privately owns the complete event list so it can admit future arrivals at their arrival times; the controller receives an immutable `RuntimeObservation`. This is the methodological core of the project: without it, any measured gain could come from information a real kernel could not have.

| Quantity | Visible to the controller? | Methodological reason |
|---|---|---|
| Current simulated time | Included | Defines the decision epoch. |
| Previously selected policy | Included | The action from the immediately preceding epoch (None at the first). |
| Ready processes (arrived, unfinished) | Included | PID, arrival time, known burst, remaining burst, priority. |
| Arrived / completed counts | Included | Progress without knowing the workload size. |
| Mean burst and priority spread over arrived work | Included | Aggregates restricted to processes that have already arrived. |
| Mean arrival age of ready jobs | Included | current_time – arrival_time; includes prior CPU service, not queue waiting. |
| Unarrived process data, future bursts/arrivals | Excluded | Would break deployability and fairness of the comparison. |
| Total workload size, workload identity/family | Excluded | Prevents the controller from memorising generator families. |
| Counterfactual or completed-schedule metrics | Excluded | No peeking at outcomes of policies that were not run. |

*Table 1.2: The causal observation contract (scheduler/runtime.py, rl/runtime_state.py). A paired hidden-future test asserts that changing unarrived processes leaves the observation and action prefixes bit-for-bit identical.*

## 1.4 State abstraction and learning objective

| # | Feature | Bins | Discretisation (q = 4) | Why it is causal |
|---|---|---|---|---|
| 1 | ready_count | 3 | {1}, {2–3}, {4+} | Instantaneous ready-queue contention. |
| 2 | completed_count | 3 | {0}, {1–3}, {4+} | Trace progress without the workload size. |
| 3 | median_ready_remaining_burst | 3 | ≤ q, ≤ 4q, > 4q | Remaining service demand of ready jobs. |
| 4 | mean_ready_arrival_age | 3 | ≤ q, ≤ 4q, > 4q | Age proxy for ready jobs (includes prior service). |
| 5 | arrived_work_priority_spread | 2 | zero / non-zero | Whether arrived processes have distinct priorities. |

*Table 1.3: 3 × 3 × 3 × 3 × 2 = 162 states; 162 × 4 = 648 state-action pairs.*

Each trace is one sequential episode. Between consecutive epochs the environment measures the waiting-time area $\Delta W_t$ accumulated by ready processes (including any switch interval) and returns a scaled interval reward:

$$r_t = -\Delta W_t / \max(1, q), \ \gamma = 1.0 \Rightarrow \Sigma_t r_t = -\text{TotalWaitingTime} / q$$

Because $\gamma = 1$ and the episode is finite, maximising the undiscounted return is identical to minimising total waiting time — the objective is not a proxy for it. The update is

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[ r_t + \gamma \max_{a' \in \text{visited}(s')} Q(s', a') - Q(s, a) \right], \ \alpha = 0.1$$

Terminal transitions omit the bootstrap; the first visit to a state forces a uniform random action; greedy selection and bootstrapping mask unvisited actions; and a wholly unseen evaluation state triggers an explicit, counted fallback to the causal heuristic instead of a silent zero-Q tie.

## 1.5 Methodological controls

| Control | How it is enforced |
|---|---|
| Fixed-policy baselines | FCFS, SJF, Round Robin and Priority run on the identical workloads and pay the identical switch cost. |
| Predeclared adaptive rule | A non-RL heuristic (RR → Priority → SJF → FCFS) was fixed before any test result was seen, so learning is not credited for what fixed rules already do. |
| Blinded held-out split | Final test (master seed 19301) is generated and evaluated after training; the pipeline asserts zero fingerprint overlap with training. |
| Out-of-family generalisation | poisson_arrivals is excluded from training entirely and used only at evaluation time. |
| Independent seeds | Five agents (7101–7105) are trained separately; Q-learning results average all five. |
| Paired uncertainty | Differences are paired per workload and per model seed, then resampled with a family-stratified crossed bootstrap (1,000 replicates). |
| Determinism | Evaluation is greedy and read-only; artifacts regenerate bit-for-bit from the declared seeds. |
| Separation of timing | Observation, selection, Q-update and simulator wall time are measured in separate columns and never charged to simulated time. |

*Table 1.4: Controls that keep the comparison honest.*

## 2. Overall conceptual, architectural and pipeline diagram

The system is layered so that the information boundary is structural, not conventional: the lower (blue) layer owns the future and the accounting, the upper (amber) layer only ever receives a frozen snapshot of arrived work. The controller's reply is a single policy name; it cannot touch the queue, the bursts or the event list.

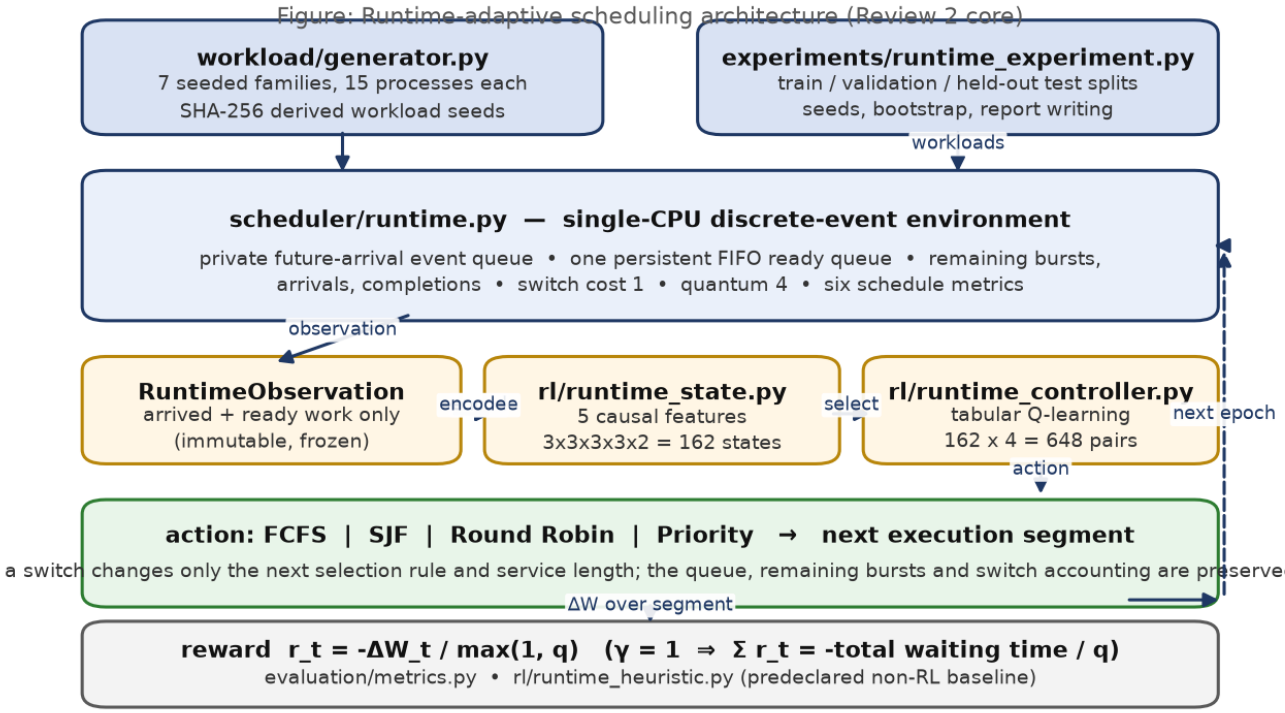



*Figure 2.1: Conceptual architecture. Blue = environment (owns future arrivals, ready queue and switch accounting); amber = controller path (arrived work only); green = the action returned to the environment.*

| Layer | Components | Responsibility |
|---|---|---|
| Data layer | workload/models.py, workload/generator.py | Defines Process/Workload objects and generates the seven seeded families with SHA-256-derived seeds. |
| Simulation layer | scheduler/runtime.py | Discrete-event loop, FIFO ready queue, remaining bursts, switch cost, schedule trace. |
| Baseline layer | scheduler/{fcfs,sjf,round_robin,priority}.py | Standalone fixed policies, preserved so runtime actions can be checked against them. |
| Perception layer | rl/runtime_state.py | Encodes an immutable observation into one of 162 causal states. |
| Decision layer | rl/runtime_controller.py, rl/runtime_heuristic.py | Learned policy (tabular Q-learning) and the predeclared non-RL rule. |
| Experiment layer | experiments/runtime_experiment.py, runtime_config.py | Splits, training, validation, held-out testing, paired bootstrap, artifact export. |
| Evaluation layer | evaluation/metrics.py, evaluation/comparison.py | Metric definitions and paired/bootstrap comparison helpers. |

*Table 2.1: Architectural layers and what each one owns.*

### 2.1 Runtime control flow

- The environment advances to the next decision epoch (a dispatch point, a completion, or a Round-Robin quantum boundary) and admits every arrival at or before that instant in (arrival_time, pid) order.
- It freezes an observation and passes it to the controller; the controller returns one of four policy names.
- If the selected process differs from the running PID, one unit of switch cost is charged before the segment starts; arrivals during that interval are queued but the dispatch already chosen is not reconsidered.
- The segment runs (Round Robin bounded by the quantum, others to completion); endpoint arrivals are admitted first, then an unfinished Round-Robin process is appended to the tail.
- The environment measures the waiting-time area accumulated by ready processes, converts it to the interval reward, and hands it back to the controller for the Q update.

- A switch changes only the next selection rule and service length — the queue, the remaining bursts and the switch accounting are preserved across switches.

## 3. Description of the major modules

Six modules carry the whole system: two build and run the simulated machine, two implement the competing controllers, one is the non-learning reference rule, and one orchestrates the experiment and its statistics. Each is described below with its contract; §4 gives the workflow diagram for each.

| ID | Module | Files | Input → output | Responsibility | Status |
|---|---|---|---|---|---|
| M1 | Workload generator and dataset builder | workload/generator.py, workload/models.py | family config + seed → validated Workload with fingerprint | Generates the seven synthetic families, validates every process (positive burst, non-negative arrival, priority bounds) and records split/seed/fingerprint metadata. | Implemented |
| M2 | Discrete-event runtime environment | scheduler/runtime.py | Workload + controller + (quantum, switch cost) → schedule trace and six metrics | Owns the private event list, the single FIFO ready queue, remaining bursts, switch-cost accounting, the causal observation and the interval reward. | Implemented |
| M3 | Causal observation and state encoder | rl/runtime_state.py | RuntimeObservation → state index in [0, 162) | Extracts five causal features, discretises them relative to the quantum and flattens them in mixed-radix order. Accepts an observation only — never a Workload. | Implemented |
| M4 | Sequential Q-learning controller | rl/runtime_controller.py | observation (encode) / reward (update) → policy action; Q table and visit counts | Tabular Q-learning over 648 pairs: ε-greedy exploration, first-visit forcing, unvisited-action masking, read-only greedy evaluation and a counted heuristic fallback. | Implemented |
| M5 | Predeclared heuristic controller | rl/runtime_heuristic.py | RuntimeObservation → policy action | Non-RL adaptive rule (RR → Priority → SJF → FCFS) sharing the same observation contract, declared before any test result was inspected. | Implemented |
| M6 | Experiment orchestration, metrics and reporting | experiments/runtime_experiment.py, evaluation/metrics.py | configuration + trained models → CSV/JSON artifacts, paired CIs, markdown report | Builds disjoint splits, trains five agents, validates, runs the held-out test, computes paired bootstrap intervals and writes every artifact. | Implemented |

*Table 3.1: Major modules, contracts and current status.*

### M1 — Workload generator and dataset builder

Seven frozen family configurations (burst distribution and range, arrival pattern, priority pattern) are turned into workloads of 15 processes. Each workload seed is derived with SHA-256 from (master seed, split tag, family index, repetition), so the whole dataset is reproducible from three integers and no external data is required. The generator validates every process and the pipeline records a workload fingerprint for split-disjointness checks.

### M2 — Discrete-event runtime environment

The simulator is deliberately the only component that knows the future. It advances time to the next epoch, maintains one persistent FIFO ready queue, tracks remaining bursts, charges one unit whenever the running PID changes, and produces the schedule trace from which all metrics are computed. Running it with a constant action reproduces the standalone FCFS, SJF, Round Robin and Priority implementations exactly — a property that is asserted by tests, not merely claimed.

### M3 — Causal observation and state encoder

The encoder's API accepts a RuntimeObservation and nothing else, which makes the information boundary checkable by inspection. Five features are discretised into $3 \times 3 \times 3 \times 3 \times 2 = 162$ states. Bin edges are expressed relative to the quantum so the abstraction stays meaningful if the quantum changes.

### M4 — Sequential Q-learning controller

A tabular agent over $162 \times 4$ pairs. During training it follows an episode-level $\varepsilon$ schedule, forces a random action on a state's first visit and masks unvisited actions in the greedy step and in the bootstrap. During evaluation it is greedy and read-only; an unseen state produces an explicit fallback to M5, and the number of fallbacks is reported rather than hidden.

### M5 — Predeclared heuristic controller

Round Robin if at least three jobs are ready and their mean arrival age is at least one quantum; else Priority if at least two ready jobs differ in priority; else SJF if the largest visible remaining burst is at least twice the smallest; else FCFS. It

exists so that the learned controller is compared against adaptivity itself, not only against fixed policies.

### M6 — Experiment orchestration, metrics and reporting

Builds the three splits, asserts they are fingerprint-disjoint, trains the five agents, evaluates greedily on validation and on the held-out test, computes paired differences with a family-stratified crossed bootstrap, and writes every metric table, Q value, visit count, coverage figure and the illustrative trace to disk.

## 4. Workflow diagram for each major module

Each module below is shown as its own workflow: the numbered steps are the order of execution inside that module, and the arrows are the data passed between them.

### M1. Workload generator and dataset builder

`workload/generator.py -> workload/models.py`

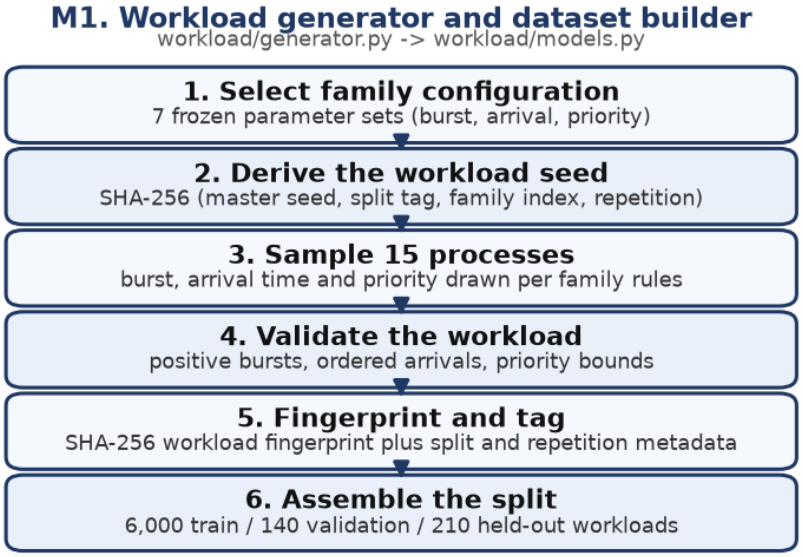



*Figure 4.1: Workflow of Workload generator and dataset builder.*

| # | Step | Detail |
|---|---|---|
| 1 | Select family configuration | 7 frozen parameter sets (burst, arrival, priority) |
| 2 | Derive the workload seed | SHA-256 (master seed, split tag, family index, repetition) |
| 3 | Sample 15 processes | burst, arrival time and priority drawn per family rules |
| 4 | Validate the workload | positive bursts, ordered arrivals, priority bounds |
| 5 | Fingerprint and tag | SHA-256 workload fingerprint plus split and repetition metadata |
| 6 | Assemble the split | 6,000 train / 140 validation / 210 held-out workloads |

### M2. Discrete-event runtime environment

`scheduler/runtime.py - one iteration per decision epoch`

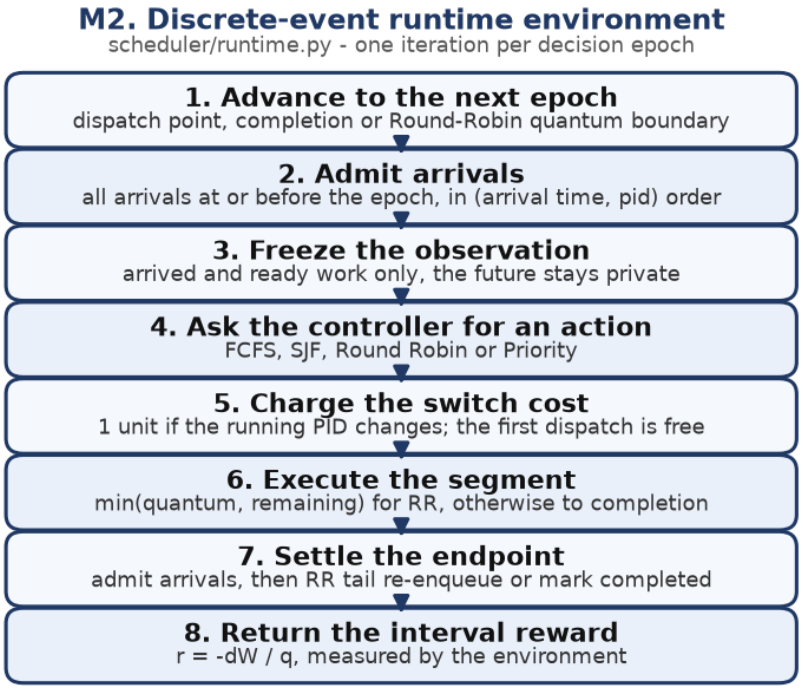



*Figure 4.2: Workflow of Discrete-event runtime environment.*

| # | Step | Detail |
|---|---|---|
| 1 | Advance to the next epoch | dispatch point, completion or Round-Robin quantum boundary |
| 2 | Admit arrivals | all arrivals at or before the epoch, in (arrival time, pid) order |
| 3 | Freeze the observation | arrived and ready work only, the future stays private |
| 4 | Ask the controller for an action | FCFS, SJF, Round Robin or Priority |
| 5 | Charge the switch cost | 1 unit if the running PID changes; the first dispatch is free |
| 6 | Execute the segment | min(quantum, remaining) for RR, otherwise to completion |
| 7 | Settle the endpoint | admit arrivals, then RR tail re-enqueue or mark completed |
| 8 | Return the interval reward | r = -dW / q, measured by the environment |

## M3. Causal observation and state encoder

`rl/runtime_state.py`

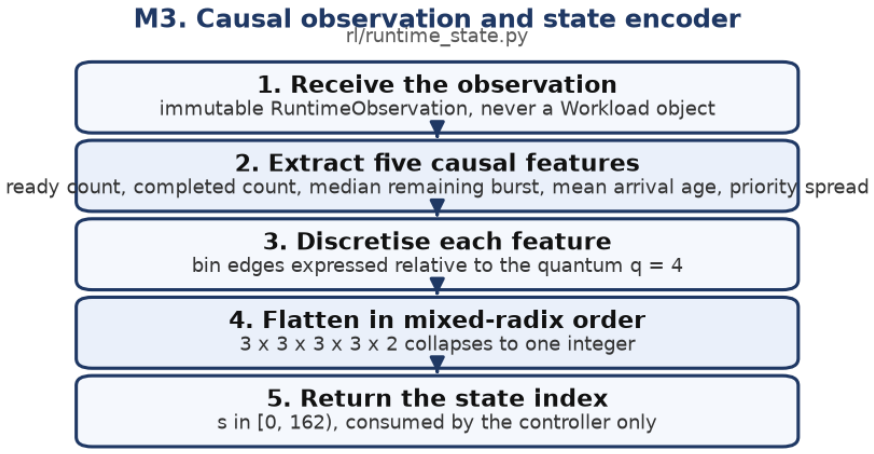



*Figure 4.3: Workflow of Causal observation and state encoder.*

| # | Step | Detail |
|---|---|---|
| 1 | Receive the observation | immutable RuntimeObservation, never a Workload object |
| 2 | Extract five causal features | ready count, completed count, median remaining burst, mean arrival age, priority spread |
| 3 | Discretise each feature | bin edges expressed relative to the quantum q = 4 |
| 4 | Flatten in mixed-radix order | 3 x 3 x 3 x 3 x 2 collapses to one integer |
| 5 | Return the state index | s in [0, 162), consumed by the controller only |

## M4. Sequential Q-learning controller

`rl/runtime_controller.py`

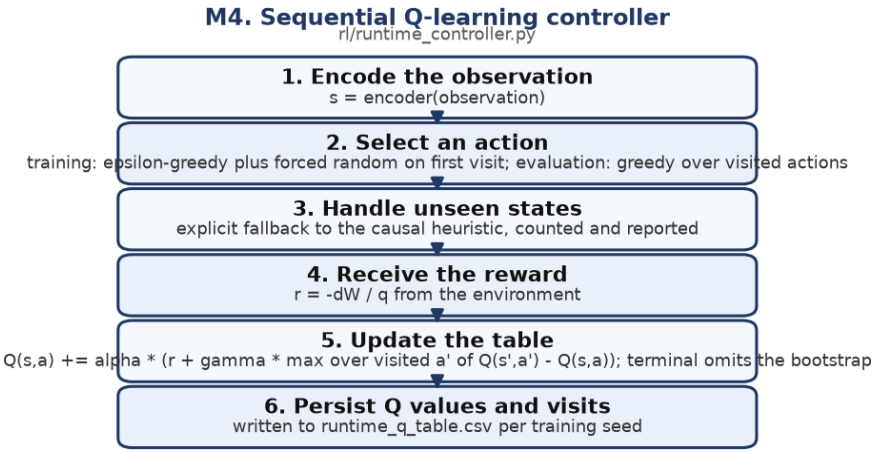



*Figure 4.4: Workflow of Sequential Q-learning controller.*

| # | Step | Detail |
|---|---|---|
| 1 | Encode the observation | s = encoder(observation) |
| 2 | Select an action | training: epsilon-greedy plus forced random on first visit; evaluation: greedy over visited actions |
| 3 | Handle unseen states | explicit fallback to the causal heuristic, counted and reported |
| 4 | Receive the reward | r = -dW / q from the environment |
| 5 | Update the table | Q(s,a) += alpha * (r + gamma * max over visited a' of Q(s',a') - Q(s,a)); terminal omits the bootstrap |
| 6 | Persist Q values and visits | written to runtime_q_table.csv per training seed |

## M5. Predeclared heuristic controller

`rl/runtime_heuristic.py - fixed rules, no learning`

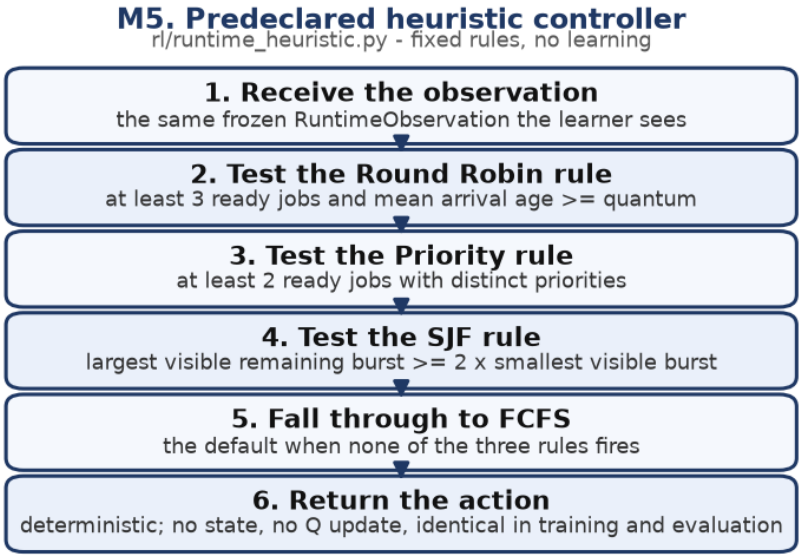



*Figure 4.5: Workflow of Predeclared heuristic controller.*

| # | Step | Detail |
|---|---|---|
| 1 | Receive the observation | the same frozen RuntimeObservation the learner sees |
| 2 | Test the Round Robin rule | at least 3 ready jobs and mean arrival age >= quantum |
| 3 | Test the Priority rule | at least 2 ready jobs with distinct priorities |
| 4 | Test the SJF rule | largest visible remaining burst >= 2 x smallest visible burst |
| 5 | Fall through to FCFS | the default when none of the three rules fires |
| 6 | Return the action | deterministic; no state, no Q update, identical in training and evaluation |

## M6. Experiment orchestration, metrics and reporting

`experiments/runtime_experiment.py, evaluation/metrics.py`

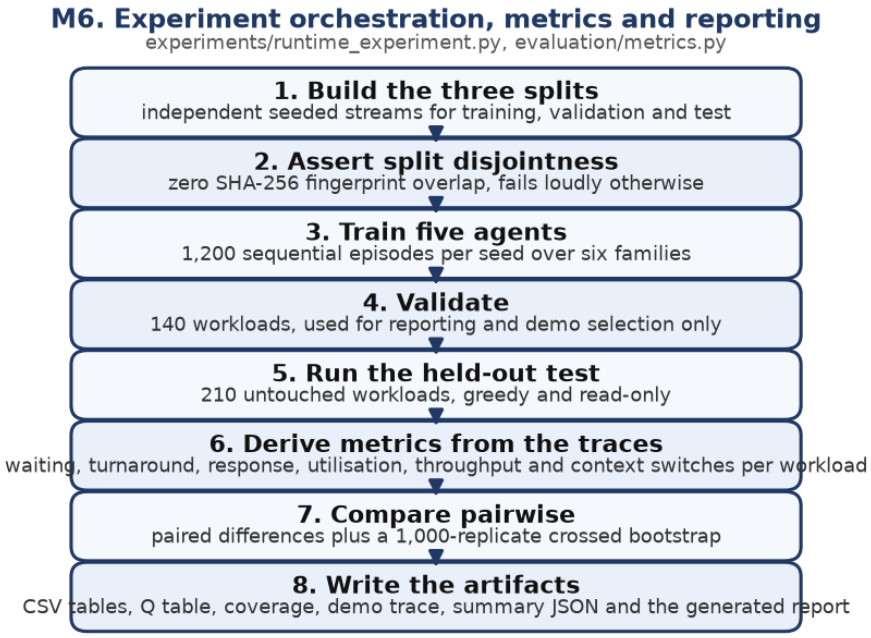



*Figure 4.6: Workflow of Experiment orchestration, metrics and reporting.*

| # | Step | Detail |
|---|---|---|
| 1 | Build the three splits | independent seeded streams for training, validation and test |
| 2 | Assert split disjointness | zero SHA-256 fingerprint overlap, fails loudly otherwise |
| 3 | Train five agents | 1,200 sequential episodes per seed over six families |
| 4 | Validate | 140 workloads, used for reporting and demo selection only |
| 5 | Run the held-out test | 210 untouched workloads, greedy and read-only |
| 6 | Derive metrics from the traces | waiting, turnaround, response, utilisation, throughput and context switches per workload |
| 7 | Compare pairwise | paired differences plus a 1,000-replicate crossed bootstrap |
| 8 | Write the artifacts | CSV tables, Q table, coverage, demo trace, summary JSON and the generated report |

# 5. Dataset details and data collection procedure

## 5.1 Data collection procedure

No external or personal data is used. The dataset is synthetic and generated entirely by the project's own generator, so it is reproducible from three integers and can be regenerated on any machine. The procedure is:

| Step | Action | Detail |
|---|---|---|
| D1 | Fix seven family configurations | burst distribution and range, arrival pattern/window/rate, priority pattern — declared once in config.py and frozen in the experiment configuration. |
| D2 | Derive one seed per workload | workload_seed = SHA-256(master seed, split tag, family index, repetition); no global RNG state is shared between workloads. |
| D3 | Generate 15 processes per workload | burst, arrival time and priority are drawn per the family rules, then validated (positive burst, non-negative arrival, priority within bounds). |
| D4 | Assign workloads to splits | 6,000 training workloads (6 families × 1,200 episodes × 5 seeds), 140 validation (7 × 20) and 210 held-out test (7 × 30). |
| D5 | Fingerprint and check disjointness | each workload gets a SHA-256 fingerprint; the pipeline asserts zero overlap between training, validation and test. |
| D6 | Store only what is needed | metrics, manifest (split/seed/fingerprint/process count) and compact summaries are committed; the large per-decision event logs are regenerated by the CLI and gitignored. |
| D7 | Regenerate on demand | `python main.py runtime-experiment` rebuilds every workload and every metric from the declared seeds. |

*Table 5.1: Data collection procedure.*

## 5.2 Dataset inventory

| Split | Families | Repetitions | Workloads | Processes | Master seed |
|---|---|---|---|---|---|
| Training (sequential episodes) | 6 families | 1,200 episodes × 5 seeds | 6,000 | 90,000 | 7101–7105 |
| Validation | 7 families | 20 repetitions | 140 | 2,100 | 8201 |
| Final held-out test | 7 families | 30 repetitions | 210 | 3,150 | 19301 |

*Table 5.2: Dataset inventory. Every workload has exactly 15 processes; poisson_arrivals is excluded from training and used only at evaluation time.*

## 5.3 Family specification

| Family | Description | Burst range | Burst law | Arrivals | Priority |
|---|---|---|---|---|---|
| short_jobs | Short-job-heavy: 80% of bursts drawn from 1-5 time units. | 1–50 | bimodal | uniform | 1–5 |
| long_jobs | Long-job-heavy: 20% of bursts drawn from 1-5 time units. | 1–50 | bimodal | uniform | 1–5 |
| mixed | Uniform mix of bursts over the full 1-50 range. | 1–50 | uniform | uniform | 1–5 |
| cpu_bursty | Long CPU bursts (20-50) with dense uniform arrivals. | 20–50 | uniform | uniform | 1–5 |
| priority_skewed | Uniform bursts; 70% of priorities lie in the 1-2 band. | 1–50 | uniform | uniform | 1–5 |
| staggered_interactive | One long process at time zero plus 14 short processes arriving at integer gaps of 4-6 time units. | 25–50 | staggered_interactive | staggered | 1–1 |
| poisson_arrivals | Held-out evaluation condition: Poisson arrivals (rate 0.5) and uniform bursts from 1-20. | 1–20 | uniform | poisson | 1–5 |

*Table 5.3: The seven declared workload families (15 processes each).*

## 5.4 Measured dataset statistics

| Family | Workloads | Processes | Burst (mean ± SD) | Mean arrival | Mean total work | High-priority share |
|---|---|---|---|---|---|---|
| short jobs | 30 | 450 | 8.5 ± 12.1 | 9.6 | 128 | 40% |
| long jobs | 30 | 450 | 22.2 ± 15.4 | 9.6 | 333 | 41% |
| mixed | 30 | 450 | 24.7 ± 14.3 | 10.0 | 370 | 41% |
| cpu bursty | 30 | 450 | 35.0 ± 9.0 | 7.8 | 525 | 38% |
| priority skewed | 30 | 450 | 26.3 ± 14.6 | 10.0 | 395 | 82% |
| staggered interactive | 30 | 450 | 4.3 ± 9.1 | 35.4 | 65 | 100% |
| poisson arrivals | 30 | 450 | 10.7 ± 5.6 | 13.5 | 160 | 44% |
| All families | 210 | 3150 | 18.8 ± 15.8 | 13.7 | 282 | 55% |

*Table 5.4: Statistics of the 210 held-out workloads (3,150 processes), recomputed with the project's generator at report-build time — these are properties of the actual dataset, not of the configuration alone. High priority means priority ≤ 2.*

Generated dataset profile — 210 held-out workloads x 15 processes = 3,150 processes (rebuilt from the declared seeds)

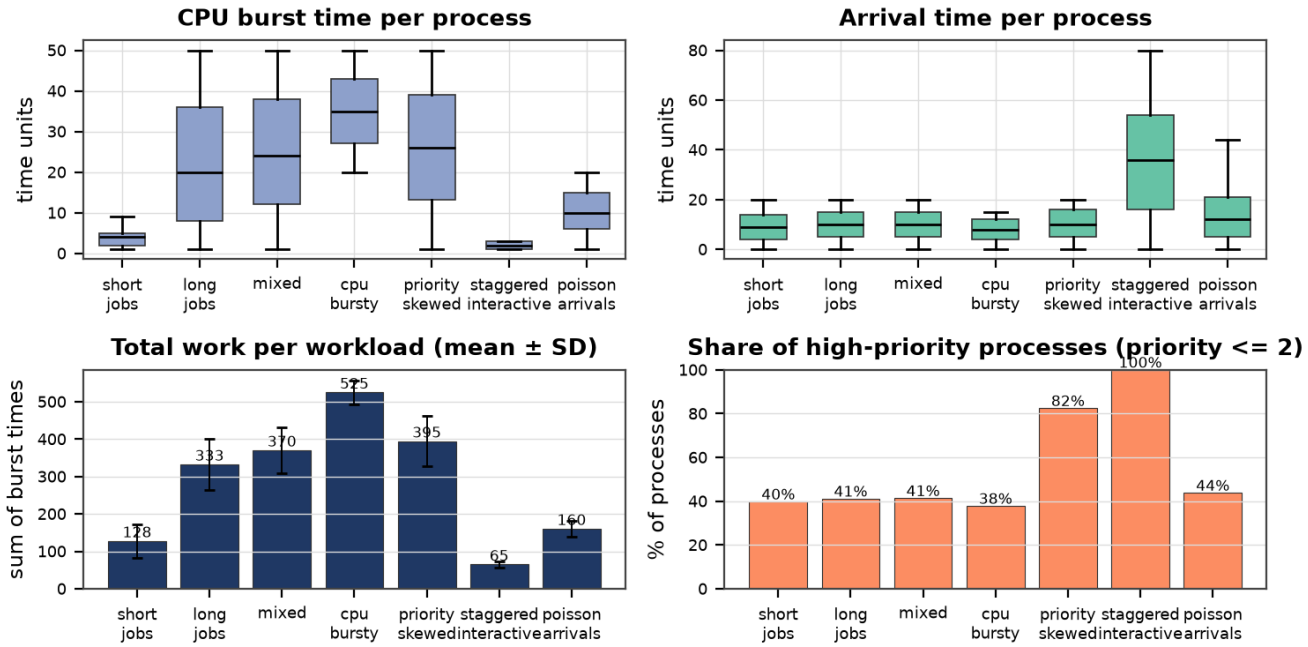



*Figure 5.1: Profile of the generated held-out dataset: burst-time and arrival-time spread per family, total work per workload and high-priority share.*

## 5.5 Data integrity and assumptions

- **Reproducibility:** seeds are derived with SHA-256, so a workload is a pure function of (master seed, split tag, family index, repetition).
- **Disjointness:** the pipeline fails loudly if any fingerprint appears in two splits (current run: overlap = false, 6,000 distinct training fingerprints).
- **Burst-knowledge assumption:** exact burst lengths become known when a process arrives. This is the assumption already required by standalone SJF; no burst predictor is included.
- **Storage discipline:** per-decision event logs are large and are regenerated by the CLI, so they are gitignored; the manifest, metrics, Q tables and summaries are committed.
- **No personal or licensed data:** the dataset is entirely synthetic, so there are no privacy or licensing constraints.

# 6. Tools, technologies, algorithms and frameworks used

## 6.1 Tools and technologies

| Category | Item | Role in the project |
|---|---|---|
| Language | Python 3.11.2 | Whole implementation, CLI and report generator |
| Numerics | NumPy 2.4.6 | Q table as an array, bootstrap resampling |
| Data handling | pandas 3.0.6 | Metric aggregation and CSV artifact export |
| Plotting | Matplotlib 3.11.2 | All figures in this report |
| Testing | pytest 9.1.1 | 273 automated checks |
| PDF generation | ReportLab 5.0.1 | Renders this report from the committed artifacts |
| Hashing | hashlib (SHA-256, stdlib) | Seed derivation and workload fingerprints |
| Standard library | argparse, dataclasses, json, pathlib | Configuration objects, CLI, artifact serialisation |
| Version control | Git / GitHub | Branch-per-change workflow, pull-request review |
| Environment | Linux 6.1 (x86-64), venv + pip | Development, experiment runs and report builds |

*Table 6.1: Tools and technologies. No deep-learning framework is used — the controller is deliberately tabular so it stays auditable.*

## 6.2 Algorithms

| Algorithm | Definition | Where it is used |
|---|---|---|
| FCFS (action 0) | Dispatch the head of the ready queue, non-preemptively. | Baseline and runtime action; after an RR requeue it follows live FIFO order. |
| SJF (action 1) | Smallest remaining burst first, non-preemptive, ties by ready order. | Baseline and runtime action; optimal for mean waiting time under known bursts. |
| Round Robin (action 2) | Head of the queue for min(quantum, remaining); unfinished jobs go to the tail after endpoint arrivals. | Baseline and runtime action; bounds response time. |
| Priority (action 3) | Highest static priority first, non-preemptive, ties by ready order. | Baseline and runtime action; lower numeric value = higher priority. |
| Tabular Q-learning | $Q(s,a) \mathrel{+}= \alpha[r + \gamma \max_{\text{visited}} Q(s',\cdot) - Q(s,a)]$ with $\varepsilon$-greedy exploration and unvisited-action masking. | The learned runtime controller (162 states × 4 actions). |
| Predeclared heuristic | RR → Priority → SJF → FCFS cascade over the same observation. | Non-learning adaptive reference. |
| Crossed bootstrap | Resample workloads within family and model seeds independently; 1,000 replicates. | Paired 95% confidence intervals for every comparison. |
| Discrete-event simulation | Event-list advancement with a persistent FIFO ready queue and switch-cost charging. | The machine being controlled. |

*Table 6.2: Algorithms implemented and their role.*

## 6.3 Controller pseudocode

```
SELECT_ACTION(obs):                    # training: epsilon-greedy; evaluation: greedy
    s <- encode(obs)                   # 5 causal features -> state in [0, 162)
    if evaluating:
        if no visited action in s:     # wholly unseen state
            fallback_count += 1
            return HEURISTIC(obs)      # predeclared rule, reported not hidden
        return argmax over visited actions of Q[s, a]
    if first_visit(s) or rand() < epsilon(episode):
        return uniform_random_action()
    return argmax over visited actions of Q[s, a]

UPDATE(s, a, r, s', terminal):
    target <- r if terminal else r + gamma * max over visited actions of Q[s', .]
    Q[s, a] <- Q[s, a] + alpha * (target - Q[s, a])
    visits[s, a] <- visits[s, a] + 1

HEURISTIC(obs):                        # predeclared, no learning
    if ready_count >= 3 and mean_arrival_age >= quantum:      return Round Robin
    if >= 2 ready jobs have distinct priorities:              return Priority
    if max(remaining_burst) >= 2 * min(remaining_burst):      return SJF
    return FCFS
```

## 6.4 Frameworks and why none of the heavy ones

The controller is a 162 × 4 table rather than a neural network, so no deep-learning framework is needed. This is a deliberate methodological choice: every Q value, visit count and fallback can be inspected directly in the committed artifacts, which is what makes claims about state coverage and unseen-state behaviour verifiable instead of anecdotal.

# 7. Experimental plan and evaluation metrics

## 7.1 Experimental plan

| Step | Action | Detail |
|---|---|---|
| P1 | Fix the design before looking at results | Families, splits, seeds, quantum (4), switch cost (1), learning rates and the heuristic rule are all declared in configuration and committed. |
| P2 | Train five independent agents | Seeds 7101–7105; 1,200 sequential episodes each on six families (poisson_arrivals withheld). |
| P3 | Validate once | 140 workloads (seed 8201) used only for reporting and for the predeclared illustrative-demo rule — never for tuning. |
| P4 | Test once, held out | 210 untouched workloads (seed 19301); the fixed policies and the heuristic are deterministic (one run each), Q-learning is evaluated by all five models greedily and read-only. |
| P5 | Compare every method against every other | Paired per-workload differences, plus independent resampling of the five model seeds. |
| P6 | Quantify uncertainty | Family-stratified crossed bootstrap, 1,000 replicates, seed 104729; 95% intervals on every headline comparison. |
| P7 | Publish artifacts | Metric tables, Q values, visit counts, coverage, manifest, demo trace and summary JSON are written to results/runtime/ and regenerated by one command. |

*Table 7.1: The experimental plan, in execution order.*

## 7.2 Evaluation metrics

| Metric | Definition | Why it is measured |
|---|---|---|
| Mean waiting time | mean over processes of (first service start – arrival) | Primary metric — it is exactly the quantity the reward minimises. |
| Mean turnaround time | mean over processes of (completion – arrival) | End-to-end latency experienced by a job. |
| Mean response time | mean over processes of (first dispatch – arrival) | Interactivity; this is what Round Robin buys. |
| CPU utilisation (%) | busy time / total elapsed time × 100 | Cost of switching and idling. |
| Throughput | processes completed per unit simulated time | Work completed per unit time. |
| Context switches | number of running-PID changes | Overhead proxy; switch cost is charged for every method. |
| Policy switches per trace | decisions where the chosen policy differs from the previous | How much the controller actually adapts. |
| State-action coverage | visited (s, a) pairs / 648, unseen-state fallbacks | Whether the learned table is actually populated where it is used. |

*Table 7.2: Evaluation metrics.*

## 7.3 Statistical procedure

Comparisons are paired: for each workload the difference (target – reference) is computed on the same workload, and for Q-learning the model seed is resampled independently from the workloads (a crossed bootstrap). Resampling is stratified by workload family, and 1,000 replicates are drawn with seed 104729. An interval that excludes zero on the favourable side is treated as a real difference on this benchmark; an interval that contains zero is reported as inconclusive rather than as a win.

## 7.4 Results obtained with this plan

| Method (210 held-out workloads) | Mean waiting | Mean turnaround | Mean response | CPU util % | Throughput | Ctx switches | Policy switches |
|---|---|---|---|---|---|---|---|
| FCFS | 127.797 ± 79.81 | 146.612 ± 90.51 | 127.797 ± 79.81 | 92.24 ± 6.15 | 0.077 ± 0.056 | 14.00 ± 0.00 | 0.00 ± 0.00 |
| SJF | 93.345 ± 68.78 | 112.161 ± 79.39 | 93.345 ± 68.78 | 92.24 ± 6.15 | 0.077 ± 0.056 | 14.00 ± 0.00 | 0.00 ± 0.00 |
| Round Robin | 215.729 ± 172.82 | 234.544 ± 183.45 | 26.220 ± 9.15 | 78.22 ± 1.90 | 0.068 ± 0.054 | 74.77 ± 40.70 | 0.00 ± 0.00 |
| Priority | 129.207 ± 80.81 | 148.022 ± 91.53 | 129.207 ± 80.81 | 92.24 ± 6.15 | 0.077 ± 0.056 | 14.00 ± 0.00 | 0.00 ± 0.00 |
| Causal heuristic | 203.404 ± 159.23 | 222.220 ± 169.85 | 40.070 ± 17.05 | 80.48 ± 2.22 | 0.071 ± 0.058 | 67.37 ± 39.49 | 3.21 ± 0.67 |
| Runtime Q-learning | 104.789 ± 71.43 | 123.604 ± 81.86 | 86.341 ± 59.82 | 90.84 ± 7.04 | 0.076 ± 0.054 | 17.32 ± 4.66 | 4.73 ± 2.83 |

*Table 7.3: Held-out final test (master seed 19301, quantum 4, switch cost 1). Mean ± SD across the 210 workloads; Q-learning also averages the five model seeds.*

Held-out final test: 210 workloads (seed 19301), 5 Q-learning model seeds

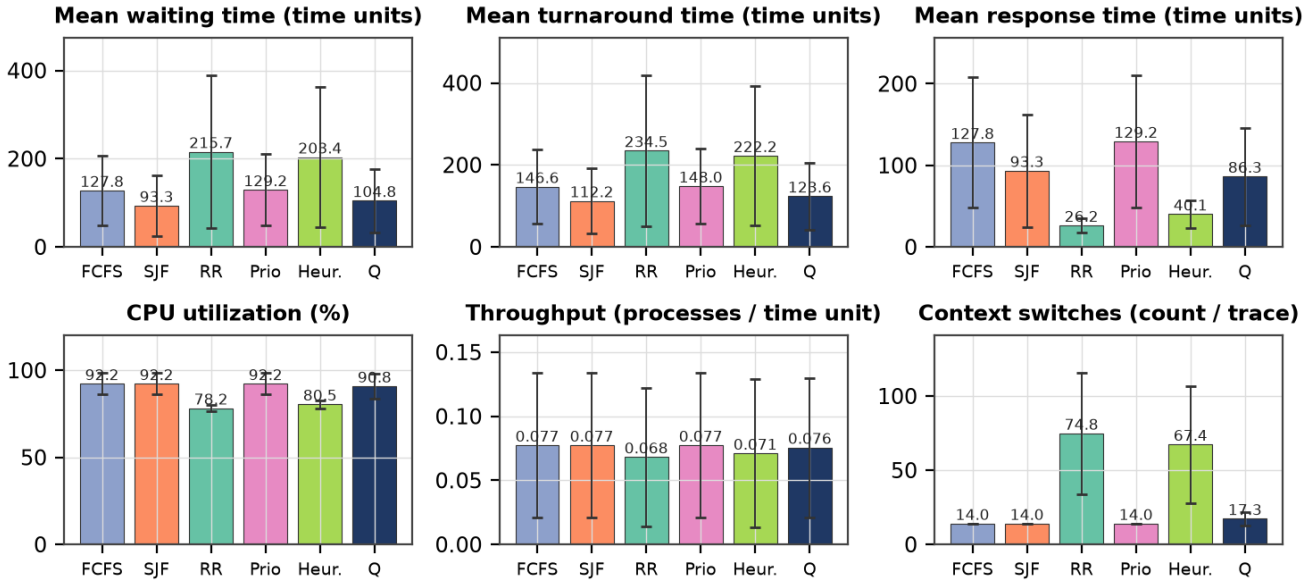



*Figure 7.1: Six metrics on the held-out test set (error bars = SD across workloads).*

5 independent agents x 1,200 sequential episodes (6,000 training workloads)

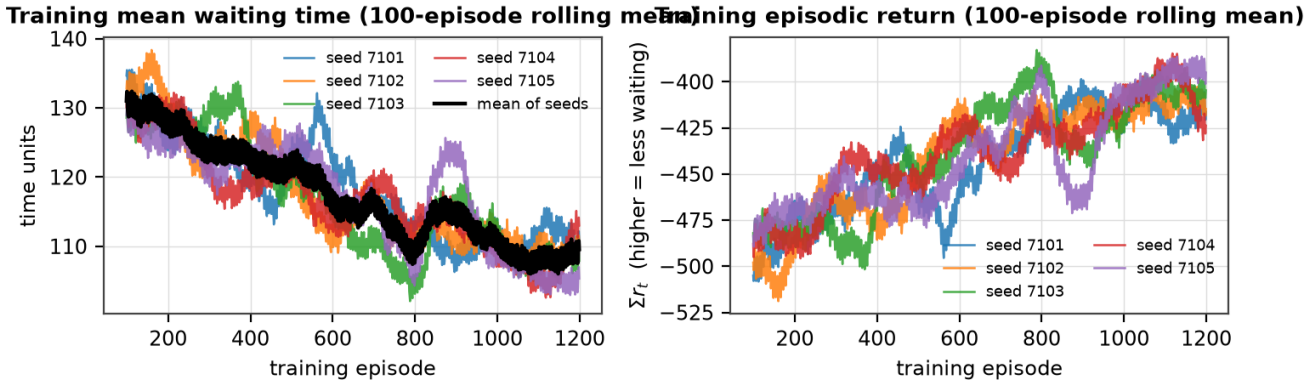



*Figure 7.2: Sequential training: rolling mean waiting time and episodic return (100-episode window) for each of the five agents.*

| Target | Reference | Metric | Mean difference | 95% bootstrap CI | Reading |
|---|---|---|---|---|---|
| Runtime Q | SJF | waiting time | +11.443 | [+5.022, +19.297] | favours reference |
| Runtime Q | SJF | response time | -7.004 | [-9.541, -4.305] | favours target |
| Runtime Q | SJF | context switches | +3.322 | [+2.209, +4.555] | favours reference |
| Runtime Q | SJF | CPU util % | -1.402 | [-1.843, -1.005] | favours reference |
| Runtime Q | Heuristic | waiting time | -98.616 | [-106.097, -90.192] | favours target |
| Runtime Q | Heuristic | response time | +46.272 | [+43.472, +49.100] | favours reference |
| Runtime Q | Heuristic | context switches | -50.050 | [-51.929, -48.066] | favours target |
| Runtime Q | Heuristic | CPU util % | +10.354 | [+9.890, +10.834] | favours target |
| RR | SJF | waiting time | +122.383 | [+117.619, +127.315] | favours reference |
| RR | SJF | response time | -67.126 | [-70.219, -64.285] | favours target |
| RR | SJF | context switches | +60.767 | [+58.995, +62.491] | favours reference |
| RR | SJF | CPU util % | -14.021 | [-14.241, -13.760] | favours reference |
| Heuristic | SJF | waiting time | +110.059 | [+106.219, +114.032] | favours reference |
| Heuristic | SJF | response time | -53.276 | [-56.026, -50.873] | favours target |

| Target | Reference | Metric | Mean difference | 95% bootstrap CI | Reading |
|---|---|---|---|---|---|
| Heuristic | SJF | context switches | +53.371 | [+51.666, +55.191] | favours reference |
| Heuristic | SJF | CPU util % | -11.756 | [-11.992, -11.518] | favours reference |
| FCFS | SJF | waiting time | +34.451 | [+32.640, +36.540] | favours reference |
| FCFS | SJF | response time | +34.451 | [+32.605, +36.440] | favours reference |
| FCFS | SJF | context switches | +0.000 | [+0.000, +0.000] | no difference |
| FCFS | SJF | CPU util % | +0.000 | [+0.000, +0.000] | no difference |

*Table 7.4: Paired differences (target – reference) with family-stratified crossed-bootstrap intervals.*

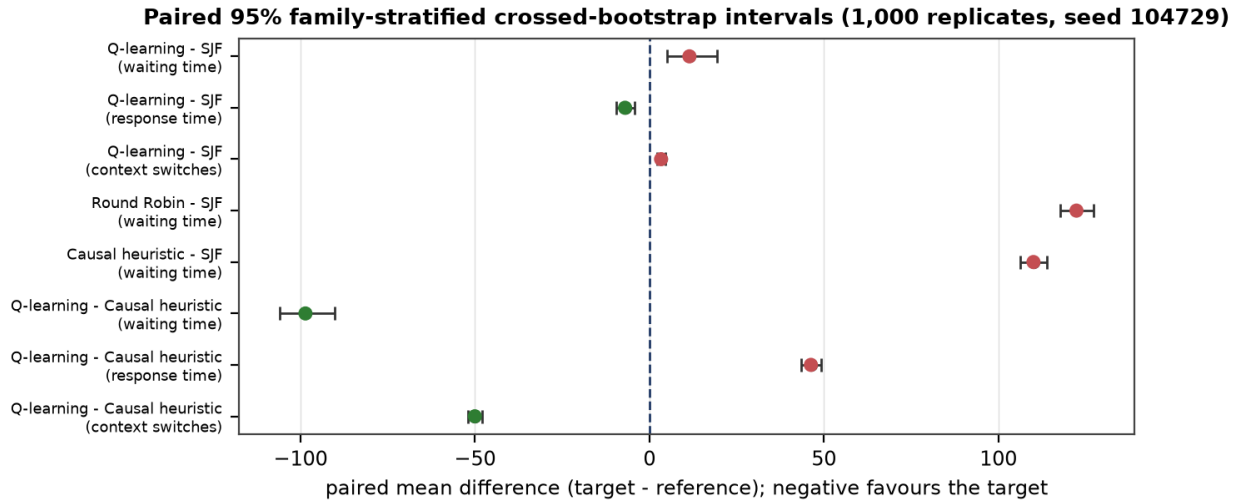



*Figure 7.3: Forest plot of the paired comparisons (green favours the target, red favours the reference).*

## 7.5 What the plan has established so far

- **Against Round Robin** the learned controller is clearly better on waiting time: -110.940 time units, obtained by differencing the two committed comparisons against SJF (Q – SJF +11.443 minus RR – SJF +122.383; both intervals exclude zero and are far apart, so the ordering is unambiguous). FCFS and Priority sit between the two, at +34.451 and +35.862 relative to SJF.
- **Against the predeclared adaptive rule** it reduces waiting time by -98.616 (95% CI [-106.097, -90.192]) — so the gain comes from learning, not merely from being adaptive.
- **Against SJF it is mixed, and the report says so:** waiting time is worse by +11.443 (95% CI [+5.022, +19.297]) while response time is better by -7.004 (95% CI [-9.541, -4.305]). Non-preemptive SJF with exact bursts and no mid-job switch cost is a very strong reference on this benchmark.
- On the held-out poisson_arrivals family the controller still lands between SJF and Round Robin, so the behaviour is not family memorisation.

# 8. Implementation progress

All six major modules are implemented, exercised by the automated suite and demonstrated with the outputs below. The requirement for Review 2 is that at least 50% of the proposed work is complete and that the completed modules are demonstrated with valid outputs; the table below records completion per module and the artifact that evidences it.

| ID | Module | Completion | Evidence artifact | Status |
|---|---|---|---|---|
| M1 | Workload generator and dataset builder | 100% | 6,350-row manifest + regenerated dataset statistics (§5.4) | Complete |
| M2 | Discrete-event runtime environment | 100% | fixed-action equivalence tests; schedule traces in final-test metrics | Complete |
| M3 | Causal observation and state encoder | 100% | 162-state table; hidden-future invariance tests | Complete |
| M4 | Sequential Q-learning controller | 100% | runtime_q_table.csv (3,240 Q values), coverage report, fallback counts | Complete |
| M5 | Predeclared heuristic controller | 100% | heuristic rows in every metric table | Complete |
| M6 | Experiment orchestration, metrics and reporting | 100% | final-test, validation, paired-bootstrap and coverage tables; generated report | Complete |
| — | Review 2 report generator (this document) | 100% | tools/build_review2_report.py | Complete |

*Table 8.1: Implementation status. Overall, the proposed Review 2 scope (methodology, simulator, controller, experiment, evaluation) is complete; the items listed in §9 are extensions beyond the proposed scope.*

## 8.1 Outputs demonstrated by each module

Each module is demonstrated by the output it actually produced in the recorded run:

| Module | Demonstrated output |
|---|---|
| M1 | 6,350 manifest rows; 210 held-out workloads (3,150 processes) regenerated from the declared seeds, with the measured burst, arrival and priority statistics of section 5 |
| M2 | 2,100 final-test metric rows produced from simulated traces; fixed-action runs reproduce the standalone FCFS, SJF, Round Robin and Priority schedules exactly in the test suite |
| M3 | 162-state encoder; 81-89 distinct states visited per agent during training |
| M4 | 3,240 Q values stored; 267-276 of 648 state-action pairs visited; 1-3 counted unseen-state fallbacks per model on the held-out test |
| M5 | heuristic rows present for all 210 held-out workloads; mean waiting 203.404 with 3.21 policy switches per trace |
| M6 | 36 paired comparisons with 95% bootstrap intervals, the coverage table, the learned-switch demo trace and the generated runtime_report.md |

*Table 8.2: Concrete output produced by each implemented module.*

## 8.2 Demonstration 1 — automated verification

| Test module | Checks | What it protects |
|---|---|---|
| test_adaptive.py | 18 | Legacy offline selector: state encoding, decisions, read-only evaluation |
| test_experiment_design.py | 4 | Frozen seeds, split sizes and configuration invariants |
| test_experiments.py | 30 | Legacy experiment pipeline, metrics and report generation |
| test_fcfs.py | 11 | FCFS ordering, idle handling and metric computation |
| test_generator.py | 7 | Workload family generation and reproducibility from seeds |
| test_metrics.py | 14 | Waiting / turnaround / response / utilisation / throughput definitions |
| test_priority.py | 14 | Priority ordering, ties and non-preemption |
| test_q_learning.py | 29 | Q updates, epsilon schedule, masking, visit counts, evaluation |
| test_reward.py | 16 | Reward composition and clipping against the reference policies |
| test_round_robin.py | 16 | Quantum boundaries, re-enqueue order and completion behaviour |
| test_runtime_experiment.py | 4 | Split disjointness, determinism, artifact regeneration |
| test_runtime_scheduler.py | 23 | Runtime environment: fixed-action equivalence, causality, reward identity |
| test_schedule_result.py | 14 | Schedule trace validation and aggregate result objects |
| test_sjf.py | 12 | SJF selection, tie-breaking and non-preemption |
| test_state.py | 24 | Offline state discretisation and mixed-radix indexing |
| test_workload_models.py | 37 | Process/workload models, validation and trace invariants |

*Table 8.3: Test inventory, 273 checks collected and executed while building this report.*

```
$ python -m pytest -q
.................................................................... [ 79%]
.....................................................                [100%]
273 passed in 12.01s
```

## 8.3 Demonstration 2 — end-to-end run output

The full pipeline (training, validation, held-out testing, artifact export) is reproduced by one command. The output below is from a verification run performed on the reporting host and matches the committed artifacts exactly.

```
$ python main.py runtime-experiment --results-dir /tmp/os-project-runtime
Causal event-driven runtime experiment
--------------------------------------
  training seeds                 : 7101, 7102, 7103, 7104, 7105
  training episodes/model        : 1200
  validation workloads           : 140
  final held-out test workloads  : 210
  final-test mean waiting time:
    FCFS                  : 127.797
    SJF                   : 93.345
    Round Robin           : 215.729
    Priority              : 129.207
    Causal heuristic      : 203.404
    Runtime Q-learning    : 104.789
  learned within-trace switches : 15 (23 decisions; runtime_learned_switch_demo.json)
  artifacts:
    training_metrics        : runtime_training_metrics.csv
    validation_metrics      : runtime_validation_metrics.csv
    final_test_metrics      : runtime_final_test_metrics.csv
    paired_comparisons      : runtime_paired_comparisons.csv
    state_action_coverage   : runtime_state_action_coverage.csv
    q_table                 : runtime_q_table.csv
    workload_manifest       : runtime_workload_manifest.csv
    learned_switch_demo     : runtime_learned_switch_demo.json
    summary                 : runtime_summary.json
    report                  : runtime_report.md
```

## 8.4 Demonstration 3 — learned behaviour and artifacts

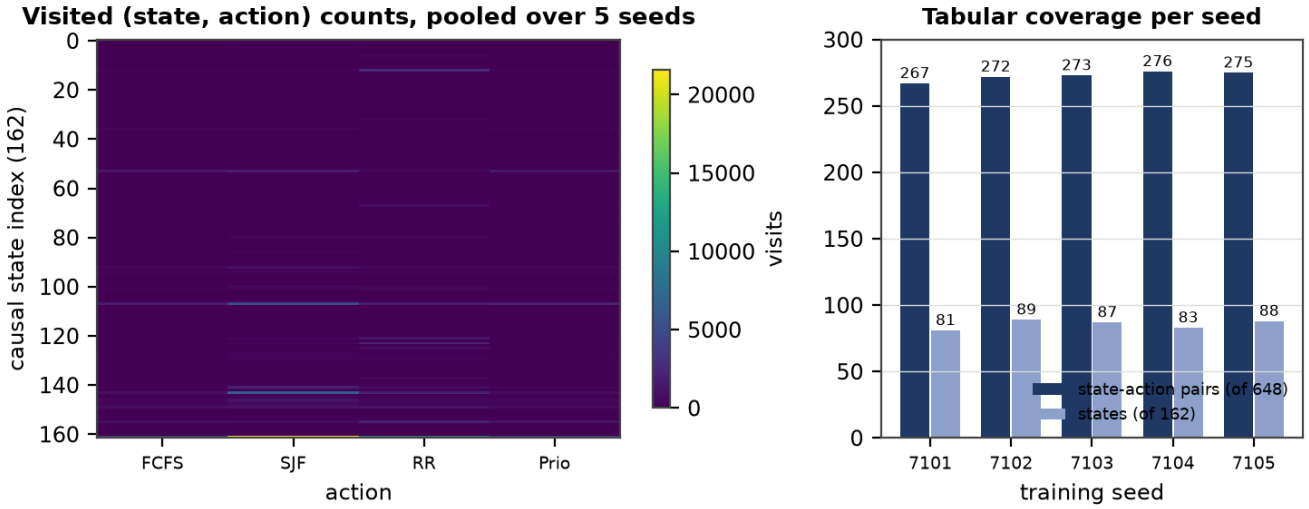



*Figure 8.1: Learned state-action visitation pooled over the five agents (left) and per-seed coverage (right). Between 267 and 276 of the 648 pairs are visited.*

The controller does switch policies inside a single trace. The example below is taken from the validation split by the predeclared rule (training seed 7103, family `staggered_interactive`, fingerprint `275409d7458331f6`): 15 switches over 23 decisions, with mean waiting time 5.933 and mean response time 2.933. It is an illustration of the mechanism, not a performance claim.

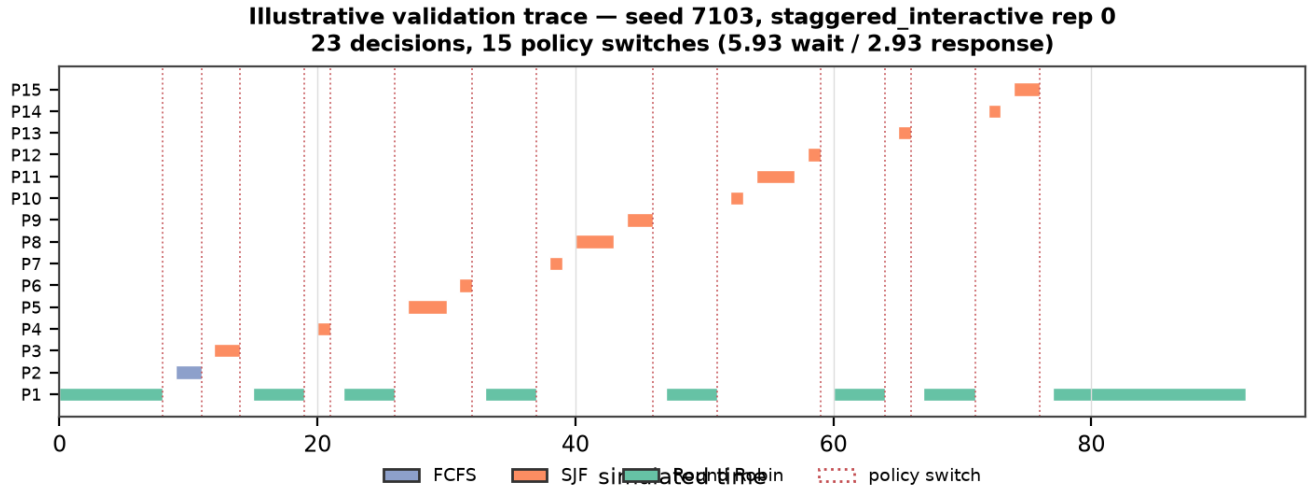



*Figure 8.2: Illustrative trace. Bars are execution segments coloured by the policy chosen for them; dotted lines mark the instants where the policy changed.*

| Artifact | Size | Contents |
|---|---|---|
| results/runtime/runtime_final_test_metrics.csv | 2,100 rows | Six metrics, switch/decision counts and timings on the held-out test |
| results/runtime/runtime_validation_metrics.csv | 1,400 rows | Same columns for the validation split |
| results/runtime/runtime_training_metrics.csv | 6,000 rows | Per-episode sequential training outcomes |
| results/runtime/runtime_paired_comparisons.csv | 36 rows | Paired differences with 95% crossed-bootstrap intervals |
| results/runtime/runtime_state_action_coverage.csv | 15 rows | Visited states/pairs, fallback counts and timings per seed |
| results/runtime/runtime_q_table.csv | 3,240 rows | Q values and visit counts per training seed |
| results/runtime/runtime_workload_manifest.csv | 6,350 rows | Split, seeds and workload fingerprints |
| results/runtime/runtime_learned_switch_demo.json | 1 trace | Observations, actions, switch times and execution trace |
| results/runtime/runtime_summary.json | — | Configuration, split fingerprints, all means and software versions |
| figures/review2/*.png | 11 figures | Every figure in this report |

*Table 8.4: Artifacts produced by the completed modules.*

# 9. Challenges encountered and plan for completing the remaining work

## 9.1 Challenges encountered

| Challenge | Why it mattered | How it was resolved |
|---|---|---|
| Keeping the controller causal | An earlier whole-workload selector saw future arrivals and counterfactual schedules, so its results could not be interpreted as a runtime controller. | Introduced an immutable RuntimeObservation, made the encoder accept only that object, and added a paired hidden-future test that fails if unarrived changes leak into the observation or action prefix. |
| Round-Robin and FCFS interaction after a requeue | Once RR appends a preempted job to the tail, 'FCFS' is ambiguous: original arrival order or live queue order? | Defined one persistent FIFO ready queue with canonical (arrival_time, pid) insertion, and specifies that FCFS follows live queue order while SJF/Priority ties preserve it. Fixed-action runs are tested against the standalone schedulers. |
| Charging switch cost fairly | If the learner pays for switching but baselines do not, the comparison is meaningless; if nothing is charged, switching is free and unrealistic. | One unit is charged whenever the running PID changes, for every method including the fixed policies; arrivals during the switch interval are queued but the already-chosen dispatch is not reconsidered. |
| Designing a reward that is the objective | A shaped reward can drift away from the metric being reported. | Used $r = -\Delta W/q$ with $\gamma = 1$, so the episodic return is exactly –total waiting time / q; a test asserts the identity. |
| Sparse coverage of the tabular space | Only ~42% of the 648 state-action pairs are ever visited, and unseen states appear during evaluation. | Kept the abstraction small, forced exploration on first visit, masked unvisited actions in greedy steps and bootstraps, and made unseen states fall back to the predeclared heuristic with the fallback count reported (1–3 decisions per model on the final test). |
| Honest reporting of a negative result | The learner does not beat non-preemptive SJF on waiting time; it would be easy to bury this. | Reported it as a headline finding with its bootstrap interval, and stated the structural reason (SJF gets exact bursts at arrival and pays no mid-job switch cost). |
| Host-dependent timing noise | Wall-clock columns make artifacts look non-deterministic across machines. | Observation, selection, Q-update and simulator time are stored in separate columns, never charged to simulated time, and excluded from all schedule comparisons. |
| Large per-decision event logs | Full causal decision logs are large and would bloat the repository. | They are written by the CLI but gitignored and regenerable from the recorded configuration and seeds; compact metrics, Q tables and manifests are committed. |

*Table 9.1: Challenges and their resolutions.*

## 9.2 Plan for completing the remaining work

| ID | Task | Gap it closes | Planned approach | Deliverable |
|---|---|---|---|---|
| W1 | Sensitivity study | All results are for q = 4 and switch cost 1. | Re-run the plan over a grid of quanta and switch costs and report where the ordering changes. | Metric tables per (q, cost) cell |
| W2 | Burst prediction | Exact bursts are assumed known at arrival. | Add an explicit predictor with measured error and re-run the controller under that noise. | Results with prediction error bands |
| W3 | Richer state abstraction | Only ~42% of pairs are visited; 162 states is coarse. | Tile coding or a small function approximator, keeping the same causal observation contract and fallback reporting. | Coverage and metric comparison vs the tabular agent |
| W4 | Multi-objective reward | Waiting time is optimised; response time is not. | Add a constrained/weighted objective and measure the waiting-response trade-off curve. | Pareto front over the two metrics |
| W5 | I/O and blocking | Processes never block, so the queue is CPU-bound only. | Model I/O bursts and blocked/wake transitions in the environment. | Extended environment plus new baselines |
| W6 | Multicore extension | Single CPU only. | Extend to N cores with per-core queues and migration cost; the controller chooses a policy per core. | Multicore metrics and overhead analysis |
| W7 | Trace-driven validation | All workloads are synthetic. | Replay a public scheduling trace through the same environment and report the gap to the synthetic results. | Trace-driven result table |

*Table 9.2: Remaining work, planned beyond the proposed Review 2 scope.*

## 9.3 Sequence for the remaining work

- **First:** W1 (sensitivity) and W2 (burst prediction) — they test whether the current conclusions are an artefact of fixed parameters or an idealised assumption.
- **Second:** W3 and W4 — they attack the two measured weaknesses directly (coarse coverage, single objective).

- **Third:** W5 and W6 — environment extensions that make the simulator more realistic, each with re-baselined comparisons.
- **Finally:** W7 — external validation on a real trace, which is the strongest available check that the synthetic ordering means anything.